Interrogación 2026-02

Sebastian Palma

Preguntas

La evaluación tiene un puntaje total de 7 puntos: 1 punto base más 6 puntos repartidos en siete preguntas. Responda todas las partes de cada pregunta según los antecedentes del caso y lo que pueda deducir de ellos. Omitir por completo una parte limita el puntaje de esa pregunta al 50 %.

Si una respuesta requiere supuestos, declárelos y úselos para explicar, contextualizar y justificar sus decisiones. La información que no se pide no suma puntaje, pero, si contiene errores conceptuales, resta puntaje.

Pregunta 1 (0,5 puntos).

Responda las dos partes:

a) Defina las siguientes métricas de Kanban: *lead time*, *touch time*, *cycle time* y tiempo de ocio (*idle time*).

b) Explique en qué se diferencian el *lead time* y el *cycle time*.

Respuestas:

1. Definiciones:
    1. **Lead time:** corresponde al tiempo transcurrido desde que una tarjeta entró al backlog de un proyecto hasta que llega a la etapa final DONE, en el caso particular de este proyecto comienza cuando parte en TO DO, se incluye el trabajo activo y las esperas posteriores.
    2. **Touch time:** esta métrica considera el tiempo en que la tarjeta esta efectivamente en trabajo activo, cuentan DOING, lo que sea de SPIKES, y CODE REVIEW, es decir, trabajo efectivo sobre la tarjeta, esperar que alguien revise algo por ejemplo no lo es.
    3. **Cycle time:** es la métrica que mide el tiempo transcurrido desde que comienza el trabajo en la tarjeta hasta que llega a DONE, comienza cuando esta DOING o SPIKES, incluye revisión posterior.
    4. **Idle time:** corresponde al tiempo sin trabajo activo dentro del intervalo medido, es el ocio dentro del cycle time, se puede medir como el *cycle time* menos el *touch time.*
2. Las dos métricas sirven para propósitos distintos, una mide cuánto espera quien solicita el trabajo hasta su término y la otra cuánto tarda el flujo una vez iniciado. Lead time mide el tiempo total de una solicitud desde que entra al proyecto, desde el TO DO, TO DO SPRINT y hasta llegar a DONE, incluyendo su ejecución en el Cycle time, lead contiene cycle time. Por ejemplo una tarjeta puede permanecer en backlog mucho tiempo y luego resolverse rápido, eso indicaría que tiene un lead alto pero un cycle bajo.

Pregunta 2 (0,25 puntos).

a) Defina *spike*.

Respuesta

1. Un SPIKE es un tipo de tarea de investigación o experimentación acotada para reducir una incertidumbre que es bloqueante para así decidir, estimar o implementar algo con suficiente contexto. Tiene un timebox, un criterio de término y un resultado accionable, la idea de esto es poder dar visibilidad a algo concreto que se busca alcanzar, llegando a una conclusión documentada o una demostración que permitiría crear o ajustar misiones/proyectos posteriores.

Pregunta 3 (0,25 puntos).

Identifique un aspecto del caso que requiera una tarjeta de *spike* y justifique por qué. El aspecto no puede tratar sobre notificaciones *push*, con ningún proveedor, porque eso corresponde a la tarjeta 2 de la tabla 1. Una respuesta basada en la tarjeta 2 obtendrá 0 %. Si propone más de un aspecto, solo se corregirá el primero.

Respuesta:

Como el equipo nunca ha implementado pagos divididos, sería interesante armar un SPIKE para evaluar la posibilidad y repartir el pago de una reserva entre varios jugadores, esto ayudaría a tener visibilidad sobre cómo se podría abordar, así reducir la incertidumbre técnica que conviene resolver antes de estimar y comprometer su desarrollo.

Esto es relevante porque pasa mucho que al reservar canchas de varias personas, a veces se generan conflictos sobre quién paga y como se le reembolsa a la persona, entonces sería mucho más transparente y cómodo para el cliente que cada integrante pague su cuota, si por ejemplo se reserva una cancha de futbol y no conoces al equipo rival, esto podría ayudar a mitigar posibles impagos, asegurándonos que el que efectivamente va a jugar pague.

Se le podría dar un timebox de 2 días hábiles a una persona, con esto se podrían revisar las capacidades del proveedor y, si lo permite, realizar una demo mínima en su ambiente de integración. La idea sería terminar con una conclusión documentada sobre la viabilidad de implementación, limitantes y preguntas pendientes que quedaron fuera del scope. El SPIKE igual podría terminar indicando que la alternativa no es viable, generar evidencia también cumple con la finalidad del mismo.

Pregunta 4 (1 punto).

Con los datos de la tabla 1, responda las tres partes:

a) Calcule el *lead time*, el *touch time*, el *cycle time* y el tiempo de ocio de cada tarjeta, en días hábiles.

b) Calcule el promedio de cada métrica con las tres tarjetas, incluida la de *spike*. Exprese cada promedio con dos decimales.

Respuestas:

1. Tabla resumen: Todos los valores calculados están en días hábiles.

Tarjeta	**Lead time**	**Touch time**	**Cycle time**	**Idle time**
1	14+4+5+3+2 = 28	5+2 = 7	5+3+2 = 10	10-7 = 3
2	20+3+3 = 26	3	3	3-3 = 0
3	11+2+4+3+2 = 22	4+2 = 6	4+3+2 = 9	9-6 = 3

2. Tabla resumen:

Métrica	**Promedio**
Lead time	(28+26+22)/3 = 25,33
Touch time	(7+3+6)/3 = 5,33
Cycle time	(10+3+9)/3 = 7,33
Idle time	(3+0+3)/3 = 2,00

Se evidencia que Touch <= Cycle <= Lead como se vió en clases y Cycle = Touch + Idle.

Asuma que una tarjeta recibe trabajo todos los días que permanece en una columna de trabajo activo. Cualquier criterio coherente con sus definiciones de la pregunta 1 y con la descripción de las columnas es válido si lo explica, incluida la columna en que comienza cada métrica. Además, el *lead time* y el *cycle time* deben comenzar en columnas distintas.

Pregunta 5 (1 punto).

Evalúe la propuesta de un integrante del equipo, que consiste en dedicar el 25 % del tiempo de pruebas a cada uno de estos cuatro tipos: pruebas unitarias automatizadas, pruebas de integración automatizadas, pruebas *end-to-end* (E2E) automatizadas y pruebas manuales. Responda las dos partes:

a) Identifique a qué patrón o antipatrón de pruebas visto en clases corresponde la propuesta.

b) Analícela en el contexto del caso e incluya sus *trade-offs*, es decir, qué gana y qué pierde el equipo con ella.

Puede compararla con otros patrones, pero no necesita proponer una estrategia alternativa, porque eso corresponde a la pregunta 6.

Respuestas:

1. De acuerdo a lo presentado en el enunciado tiene cara de estar emulando al antipatrón cupcake visto en clases, asigna una proporción a pruebas automatizables y otro trecho mantiene una porción importante en pruebas manuales.

2. En general reservar espacio a cuatro tipos de pruebas complementarias les permite ganar, partiendo con los unit tests para probar componentes, estas permiten aislar fallas localizadas, los de integración permitirían descubrir incompatibilidades entre componentes, los E2E verifican el recorrido visible desde el inicio de un journey hasta su completitud, sirve pero son costosos, mientras que el último, pruebas manuales permiten encontrar problemas de usabilidad o situaciones no previstas como mal usos de la aplicación.
   Sin embargo, la igualdad de porcentajes no permite reflejar el contexto real del proyecto, no responde a los riesgos particulares que enfrenta esta aplicación en términos de costo ni tiempo. El hecho de repartir un cuarto del tiempo a pruebas E2E puede rendir poco y generar diagnósticos lentos, así también con que otro resto vaya a pruebas manuales, el hecho de que sean cuatro ingenieros con tres meses restantes, hace que compitan los esfuerzos por cubrir el tema de pagos, vencimientos en caché que podrían darse por pruebas rápidas y automatizables.

Pregunta 6 (3 puntos).

Defina una estrategia de *testing* adecuada al caso y justifique cada decisión con sus *trade-offs*. La estrategia debe incluir las cinco partes:

a) Cómo distribuye el esfuerzo, en cantidad de pruebas o en tiempo, entre los tipos de prueba que propone y qué patrón visto en clases sigue, si sigue alguno.

b) Qué tipos de prueba verifican cada página, *endpoint* y *service*, y cómo verifica la integración con cada servicio de AWS y cada servicio externo. Si decide no probar algún componente, explique por qué. Puede agrupar los componentes que se prueban de la misma forma.

c) Cómo verifica el recorrido completo por las tres páginas, desde la búsqueda hasta el pago aprobado, y en qué ambiente, o por qué no lo verifica de punta a punta. Los correos de confirmación no forman parte de este recorrido.

d) En qué *pipeline* se ejecuta cada prueba automatizada, qué evento lo dispara (por ejemplo, la apertura de un *pull request*, un *merge* a la rama principal o un horario programado) y en qué posición queda respecto de las etapas de construcción y despliegue.

e) En qué momento se realiza cada prueba manual que proponga. Si no propone pruebas manuales, indíquelo.

Respuestas:

Como supuesto se adopta que la estrategia busca proteger las reglas más costosas de incumplir sin detener las funcionalidades, se priorizaría los tests sobre pagos, retenciones y disponibilidad, luego temas de notificaciones y caché. También se revisaría la ficha de las canchas por ser la página más visitada.

Se asume que los cuatro ingenieros implementan y revisan pruebas como parte de las tarjetas, que una tarjeta funcional no llega a DONE sin verificaciones acordadas para su riesgo.

1. Se propone una estrategia de testing en forma de piramide:
    1. **Pruebas Unitarias:** dedicando un 50% del tiempo a pruebas unitarias, ya que permite probar rápido componentes sueltos lo cual va de la mano con menos costo ya que se requiere menos capacidad computacional y consumo de recursos por ejecutar los pipes de pruebas unitarias.
    2. **Pruebas de Integración:** dedicar un 35% del tiempo a pruebas de integración ya que permiten proteger los contratos http, persistencia y caché que manejamos ya sea por nuestros propios servicios como por el consumo de proveedores, esto requiere cooperación entre diferentes capas, preparar servicios y preparar datos donde corresponda pero sirve para evitar que cambios en componentes unitarios rompan sus dependencias directas en un flujo o proceso.
    3. **Pruebas E2E automatizadas:** se les estima dedica un 10% del tiempo, ya que mantenerlos es mas lento y costoso, solo se dedicarían recursos para probar flujos end to end que sean críticos y bloqueantes para la operación en caso de falla.
    4. **Pruebas Manuales Exploratorias:** para las pruebas manuales se estima dedicar un 5% del tiempo para tener una comprensión de la interfaz, el flujo del usuario y como se interactúa con la aplicación.
2. En líneas generales, se adoptaría una estrategia híbrida combinando London/Mockist en pruebas unitarias y Detroit/Classical en integración, conectando los controllers, services y diferentes almacenamientos reales de prueba, se simularán proveedores cuando se necesite forzar fallas y evitar cobros innecesarios.
   Para testear cada concepto que indica el enunciado:

Páginas: Ya que el front utiliza React escribiría tests automatizados con React Testing Library para ejecutar componentes reales de cada vista y así simular respuestas http del backend, así comprobaría la interfaz sin servicios pagados, pruebas más intensivas se resuelven en el E2E. Serán ejecutables en local o en un pipe al abrir PR hacia alguna rama que vaya a deployar producto.

- Buscar canchas: Los Tests Ingresarían deporte, comuna, fecha y precio, ejecutaría la búsqueda y tendría que retornar una lista conocida o esperada, comprobaría usando los filtros existentes de la página y los resultados mostrados y trataría de hacer que me de una lista vacía para ver cómo se hace el display en la UI y sus respectivos mensajes.
- Ficha de cancha: Los Tests simularian horarios distintos para dos días y verificarían que cuando cambiemos de fecha, la página consulte y muestre los bloques correspondientes al día seleccionado, además de que simularían una retención aprobada o rechazada, probaría combinaciones de tipos de canchas con distintos pronósticos de clima revisando que la ficha siga utilizable, habría que priorizar más casos borde para esta vista porque es la más concurrida.
- Pago: Los Tests probarían teniendo una retención que sucede al pagar y simularían los diferentes escenarios como aprobación, rechazo, falta de respuesta o vencimiento, esto no con un pago real de webpay, si no a nivel de componente UI interfaz.

  Endpoints: Siguiendo la misma linea de las páginas se armarían tests automatizados de integración siguiendo el criterio de pruebas Detroit, que se puedan ejecutar en local y en un proceso de CI al levantar PR a una rama deployable. La idea sería que cada test prepare datos, lleve una solicitud a FastAPI y verificar mediante aserciones las respuestas.

- **Consultas GET /api/v1/canchas y /api/v1/canchas/{id}/bloques:** Se realizarían tests automatizados de integración con criterio Detroit, ejecutando controllers, services y postgreSQL/Redis reales de prueba. Antes de cada prueba se cargan fixtures de canchas, horarios, reservas y retenciones, al finalizar se limpian esos datos. Los tests deben comprobar que se respeten los filtros de busqueda y fechas permitidas para los diferentes bloques, usando la bbdd propia y caché según corresponda el test. Openweather en esta etapa se simulará para comprobar respuesta con o sin pronóstico y no consumir cuota real de este servicio particular.
- **Operaciones POST /api/v1/retenciones y /api/v1/pagos:** Para este caso se escribirían tests automatizados siguiendo el criterio Detroit igual, con componentes propios y almacenamiento reales de prueba. Al igual que antes las fixtures ayudarán a preparar un jugador autenticado y los bloques libres, ocupados o retenidos según diferentes tipos de canchas para cada caso. Así podremos comprobar que una retención permitida se guarde en Redis y que una solicitud rechazada no cree otra retención. Para el caso de los pagos, se verificarían las reservar en postgreSQL, las eliminaciones de retenciones y la publicación del mensaje sustituto de SQS. Para estos casos se comprobará simulando SQS y Webpay para controlar resultados y evitar cobros.

  Se omitieron las pruebas unitarias para controllers que solo comprueban delegación, como llamadas a *services*, porque las pruebas de integración HTTP aportan mayor confianza. Si los controllers aplican validaciones o transformaciones propias, esas sí se probarían.

  Servicios: Para los servicios se propone una bateria de tests unitarios automatizados tipo London. Los tests probarían cada service por separado, los datos reales y la comunicación se comprueban en las distintas pruebas de integración de endpoints y proveedores, evitando duplicar suites.

- **Busqueda, disponibilidad y clima -> BusquedaService, DisponibilidadService y ClimaService:** Se agruparon estos servicios ya que atienden al mismo scope lógico de reserva de bloque para una cancha. Los test buscarían simular almacenamiento, consultar entre otros services y OpenWeather, la idea sería

comprobar los filtros de días hoy + 7 días por ejemplo, exlclusión de bloques iniciados, reservados o retenidos. En cuanto a las cachés se verificará que un resultado vigente se devuelva sin consultar la bbdd, y que al faltar o vencer se consulte y se guarde por 5 minutos para busqueda y una hora para clima por ejemplo. Por ejemplo, un pronóstico de hace 30 minutos debe retornarse sin llamar nuevamente a la API de OpenWeather, al cumplir una hora debe consultarse nuevamente para refrescar la información.

- **Retención y pago -> RetencionService y PagoService:** Aquí se buscaría principalmente que los tests simulen disponibilidad, Redis, PostgreSQL, Webpay y SQS. Se probará por ejemplo que solo se deberían conceder retenciones en bloques disponibles. Para pagos se debe verificar cuando no hay retención inicial para cubrir gaps de pagos sin retención y otros errores sin llamar a la API de Webpay. Se debe probar la retención vigente, la creación de la reserva y eliminación de la retención. También se comprobaría que el jugador no tenga otra retención vigente y que la nueva dure diez minutos. Si el pago es rechazado o no responde, la retención debe mantenerse hasta vencer y permitir otro intento. Una aprobación posterior al vencimiento debe solicitar devolución sin crear reserva, mientras que una aprobación con retención vigente debe crearla aunque el bloque haya comenzado.
- **Notificaciones -> NotificacionService:** Los tests deberían entregar un mensaje de reserva con jugador, recinto y destinatarios definidos por prueba y simularán aceptaciones y rechazos de SES. La idea es poder mapear que ocurre cuando se concreta efectivamente una reserva y así también como se reintenta cuando falla, se reintentará inmediatamente solo el correo fallido, hasta cuatro envíos contando el primero. Si todos fallan, se comprobará con un spy que se intentó registrar el error, porque el mock de Splunk no guarda los mensajes.

Proveedores

- **RDS Y ElastiCache:** Se adoptaría un criterio Detroit para los tests pero de forma selectiva y asumiendo el costo fijo del ambiente de prueba y los cobros por solicitudes, transferencia de datos y correos enviados. La idea sería validar con los tests los casos donde se guarda y consulta una reserva además de cuando se crea, consulta o expira una retención, así podemos validar como nuestra aplicación verifica datos, permisos y conexiones reales. No es necesario una bateria extensa para estos servicios si vamos a asumir costos pero si un pool acotado de pruebas que validan estos casos importantes. Además sería oportuno hacer pruebas para validar el uso del caché para pronósticos y retenciones.
- **SQS y SES:** Para estos servicios haría tests automatizados con ambos servicios reales AWS de prueba, en criterio de testing Detroit, asumiendo los costos fijos por el ambiente de prueba. La idea sería crear un test que crea una reserva media un pago aprobado, el PagoService publicaría el mensaje en SQS y el NotificacionService lo leería enviando los correos respectivos a jugador y recinto usando direcciones de prueba. Así comprobamos que SES acepte ambos correos antes de cumplirse 60 segundos desde que se guardo la reserva. Para probar errores usaría dependencias simuladas tipo, siguiendo un criterio London, así verificamos el manejo de fallas sin provocar rechazos reales ni pagar reenvíos.
- **Webpay:** Para este proveedor se propone una estructura híbrida dependiendo de la fase de implementación en la que se encuentre el MVP, Detroit en el ambiente de integración habilitado y London para escenarios controlados donde la respuesta de la API no es el eje principal. Se crearán tests donde se realizarían pagos y devoluciones con tarjetas y credenciales del entorno de integración y se verificarán las respuestas. Este proveedor es crítico para reservas así debe tener una suite robusta para comprobar edge cases, es necesario comprobar timeouts y aprobación tras vencimiento, se deben cubrir estos casos para minimizar errores en los cobros a clientes. Es importante considerar que en ambiente de pruebas Webpay no cobra comisiones.

- **OpenWeather:** Principalmente tests con criterio London simulando pronósticos, falta de respuesta y renovación de caché. Como la falla en este servicio no genera bloqueos en reservas no es crítico para la operación, por lo que destinar costos a este proveedor no vale la pena.
- **Splunk:** Se realizarian tests con criterio London sin integración real usando mock_log_to_splunk en controllers y services. Se omitirían los logs reales por costo y menor prioridad frente a pagos y reservas.

3. Automatizaría con Cypress un recorrido en un ambiente previamente configurado de pruebas en AWS, así un jugador ya autenticado busca una cancha, abre su ficha, selecciona un bloque futuro y paga con una tarjeta aprobada de webpay de integración. Para estos propósitos usaría tanto el front como el backend, RDS y ElastiCache reales de ese ambiente y comprobaría confirmación visible, reserva persistida y eliminación de retención.
   Con esto validamos que las distintas piezas funcionen en conjunto, el mayor costo de esto se controla manteniendo pocos recorridos completos y probando rechazos, vencimientos u otros tipos de estados con pruebas unitarias y de integración. Como se indicó previamente en esta interrogación, y la estrategía de testing, la idea es dejar los E2E para los flujos críticos que en verdad son necesarios tener bien mapeados y validados, como la reserva y pago en este caso.
   En esta prueba se sustituiría OpenWeather por una respuesta simulada para evitar consumir cuota y depender de su disponibilidad; su integración real se comprueba por separado, como se indicó anteriormente. Los endpoints de nuestra aplicación sí se ejecutarán realmente, sin simular sus respuestas, para verificar la comunicación entre frontend y backend. Splunk permanecerá sustituido por el mock del SDK. Los correos quedan fuera del criterio de éxito de este E2E, según el enunciado

4. Para todos los tipos de tests propuestos en los incisos anteriores se proponen estos pipes de ejecución:

    1. **Pruebas unitarias e integración de backend sin proveedores reales:** Se ejecuta al abrir o actualizar un PR ejecutando la suite que corresponda acorde a los archivos comprometidos y sus dependencias en un proceso de CI/CD antes de construir el bundle y desplegar. En las pruebas unitarias se sustituyen las dependencias del service, incluido almacenamiento. Para cuando sean tests de integración se ejecutan los componentes propios y PostgreSQL, Redis de prueba fuera de AWS. En ambas se sustituyen Webpay, OpenWeather, SQS, SES y Splunk cuando intervengan.
    2. **Pruebas unitarias frontend y tests de vistas con backend simulado:** Se ejecutan de igual manera al abrir un PR a la rama que deploya producto antes de que se construya el bundle y deploye posteriormente, como se indicó anteriormente, las pruebas con React Testing Library.
    3. **Pruebas de integración backend con AWS y Webpay de integración reales:** Se ejecuta un pipe sobre el entorno de pruebas AWS después de hacer merge, generar el paquete y desplegarlo en el ambiente de pruebas, es decir, se prueba cuando se integra antes de lanzarse productivamente, aquí verificamos RDS, ElastiCache, SQS, SES y Webpay con lo antes descrito en los incisos anteriores. En este punto OpenWeather y Splunk permanecen sustituidos. Solo se desplegará a producción si las pruebas pasan, usando el mismo paquete probado.
    4. **E2E de las páginas:** estos pipes E2E se ejecutan al hacer merge también en un proceso de CD, se genera el bundle que se deployará en un ambiente de pruebas, se usan las dependencias antes nombradas en el apartado anterior sin simular las respuestas de nuestros endpoints, se ejecuta desde frontend y backend ya que el flujo puede romperse en ambas partes, todo esto antes de producción.

    5. **Integración real con OpenWeather:** cuando cambie el código o la configuración de esta integración, una persona iniciará el workflow del backend, después de desplegar la nueva versión al ambiente de pruebas y antes de producción. Un test automatizado hará una llamada real y comprobará que la aplicación interprete correctamente el pronóstico. Como las 1000 llamadas gratuitas diarias se comparten con producción, solo se ejecutará si queda cuota disponible; de lo contrario, se registrará que esta comprobación quedó pendiente. Las pruebas con respuestas simuladas se ejecutarán igualmente.
5. Las pruebas manuales se las tendría que repartir entre los cuatro ingenieros y serían principalmente para las pantallas modificadas, es decir, durante el desarrollo ir inspeccionando los cambios en frontend por ejemplo claridad de mensajes, navegación, estados de carga, error y presentación, o alguna inspección menor por algún caso borde no deseado en las lógicas del backend, pero solo cuando corresponda, para eso están las automatizaciones que son el core de nuestra propuesta, por tanto, sería bueno poner un gate humano entre la ejecución de tests automatizados y el despliegue de un nuevo release. La idea de esto es interceptar errores que no logren capturar los tests per se.

Utilicé Codex GPT-6.1-Sol effort Medium como apoyo para interpretar el enunciado, localizar conceptos en el material del curso, desarrollar y contrastar alternativas de testing, verificar cálculos y revisar la claridad y organización de mis respuestas. También solicité propuestas de redacción como material de trabajo.

Las respuestas finales fueron escritas por mi.